Department	Artificial Intelligence And Machine Learning
Program	Computer Science Engineering
Course Code & Course Name	MLA0402 – DEEP LEARNING
Academic Year / Batch	4th /2023
Faculty Name	Dr. K. Logu
Assignment Title	Design and Evaluation of an AI-Driven Federated Learning Analytics Compiler Using Lexical Analysis and Advanced Parsing Techniques
Date of Issue	01/09/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO1 – Apply deep generative techniques for synthetic financial transaction generation.CO2 – Evaluate generative models based on data quality and diversity.
Bloom's Taxonomy Level	BTL3 – Apply, BTL4 – Analyze, BTL5 – Evaluate, BTL6 – Create
SDG Mapping	SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities
Industry / Societal Relevance	Synthetic financial data generation for secure, privacy-preserving, and efficient financial analytics.
Group Members	192325016 - B. BHANUTEJA REDDY 192325032 – K. ABHIRAM 192325060 – H. MOHAMMED NAWAZ

Department of Computer Science and Engineering

ASSIGNMENT QUESTIONS

Course Code & Name : MLA04 – Deep Learning

Assignment Title: Financial Fraud Detection

Course Outcome(s) – CO: CO5

Bloom's Taxonomy Level: L3 – Apply; L4 – Analyse; L5 – Evaluate

SDG Mapping (SDG 1–17): SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure

Problem Statement

Implement and evaluate a deep generative model for a financial-transaction dataset using either a Boltzmann Machine-based approach or a directed generative network. Train the model to generate structured or sequential synthetic transaction data relevant to the selected dataset. Demonstrate the generation process, present representative outputs, and critically evaluate their quality, diversity, limitations, and possible improvements.

Assessment Question – Question 5 (CO5)

Implement and evaluate a deep generative model for a financial-transaction dataset using either a Boltzmann Machine-based approach or a directed generative network. Train the model to generate structured or sequential synthetic transaction data relevant to the selected dataset. Demonstrate the generation process, present representative outputs, and critically evaluate their quality, diversity, limitations, and possible improvements.

Assessment Rubric

Criteria	Max Marks	Performance Indicators
Excellent	20	Generative model is correctly implemented; generated outputs are clearly presented and critically evaluated for quality, diversity, and limitations.
Good	20	Correct implementation with reasonable generated-output evaluation.
Satisfactory	20	Basic generative model demonstration with limited evaluation.
Needs Improvement	20	Generative model is missing, incorrect, or outputs are not meaningfully evaluated.

Deliverables

1. Problem formulation and dataset description
2. Mathematical analysis and model design justification
3. Pseudocode / workflow diagram
4. Source code / notebook with clear comments

5. Implementation results, evaluation metrics, and visualizations
6. Generated outputs where applicableGitHub repository uploadShort reflection on design decisions, challenges, and learning outcome

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
SIMATS ENGINEERING

COURSE ASSIGNMENT

Financial Fraud Detection

Using a Deep Generative Model (Gaussian-Bernoulli Restricted Boltzmann Machine)

Course Code & Name: MLA04 – Deep Learning
Course Outcome: CO5 | Assessment Question 5
Bloom's Taxonomy Level: L3 – Apply; L4 – Analyse; L5 – Evaluate
SDG Mapping: SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure

---

Code of Conduct:

*I **BANDLAPALLI BHANUTEJA REDDY (192325016)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B. BHANUTEJA REDDY

1. Problem Formulation and Dataset Description

Every card-payment network processes millions of transactions a day, and each transaction carries a small footprint of numbers around it — how much was spent, how far from the customer's home address, how risky the merchant category is, how recently the same card was used, and so on. Fraud detection systems are built on top of exactly this kind of data, but two very practical problems get in the way of building and testing such systems: genuine transaction data is extremely sensitive (it cannot be freely shared, published in a report, or handed to students for coursework), and fraud itself is rare, so any dataset that is shared is almost always heavily imbalanced. This assignment tackles both problems at once by asking a generative-modelling question:

"Can a deep generative model learn the underlying joint distribution of a bank's transaction features closely enough that samples drawn from it look, behave, and are as useful for downstream fraud detection as real transactions — without ever exposing a real customer's data?"

This is the textbook setting for a Boltzmann-machine-based generative model: instead of hand-coding rules for what a fraudulent transaction looks like, we let a Restricted Boltzmann Machine (RBM) learn the energy landscape of normal vs. anomalous transactions directly from data, and then sample brand-new, synthetic transactions from that learned landscape.

1.1 Dataset — Note on Simulation

Exactly as with any real banking dataset, genuine transaction-level data is not something that can be attached to a coursework submission, so a dataset was simulated in Python (NumPy, seed = 42 for full reproducibility) so that the entire generative-modelling pipeline below could be run and evaluated end-to-end on real numbers rather than left as untested theory. The simulation deliberately builds in two underlying drivers — a customer_profile factor (a customer's normal spending behaviour: typical amount, typical balance, typical hour) and an anomaly_driver factor (unusual, fraud-like behaviour: odd hours, high-risk merchants, large distance from home) — which are blended with random noise to produce 7 realistic transaction features plus a continuous Fraud_Score. This gives a known ground-truth structure to sanity-check against once the RBM has been trained, in exactly the same spirit as the reference Smart-Agriculture report's three simulated hidden agricultural drivers.

The dataset contains 500 simulated transaction records with a fixed random seed, so running the attached script again reproduces the identical numbers, figures and tables presented in this report.

1.2 Variables in the Dataset

The dataset has 7 continuous predictor variables plus a continuous Fraud_Score (used both as a generative target and, after thresholding, as the binary fraud label for downstream evaluation). Table 1 summarises them.

Group	Variable	Unit	Simulated Range
Transaction	Transaction_Amount	USD	~5 – 360
Account	Account_Balance	USD	~50 – 12,000
Behavioural	Transaction_Hour	hour (0–23)	0 – 23
Behavioural	Days_Since_Last_Txn	days	0 – 12
Merchant	Merchant_Risk_Score	score (0–1)	0 – 1
Behavioural	Txn_Frequency_7d	count/week	0 – 13

Group	Variable	Unit	Simulated Range
Location	Distance_From_Home_km	km	0 – 95
Target	Fraud_Score	score (0–1)	0 – 1
Target (derived)	Is_Fraud	binary (0/1)	0 or 1 (~12% positive)

Table 1: Variables in the simulated financial-transaction dataset (n = 500 transactions)

A short preview of the raw simulated records is shown in Table 2, so the reader can see this is real tabular data produced by executing the code, not just a description.

Txn_ID	Amount	Balance	Hour	Merchant Risk	Distance (km)	Fraud_Score	Is_Fraud
T0001	168.82	4909.31	9.02	0.692	52.16	0.678	1
T0002	51.88	1396.08	1.71	0.190	0.02	0.399	0
T0003	156.07	7375.26	21.74	0.000	0.00	0.041	0

Table 2: Preview of raw simulated dataset (first 3 of 500 rows; full CSV has 500 rows x 10 columns)

1.3 Why a Restricted Boltzmann Machine Is the Right Tool Here

The assignment brief explicitly offers a choice between a Boltzmann-machine-based approach and a directed generative network such as a VAE or GAN. A simpler route would be to train a directed network (e.g. a Variational Autoencoder), but a Boltzmann machine was chosen deliberately for this problem for three reasons. First, an RBM is an undirected, energy-based model — it does not need a decoder network to be differentiable end-to-end, so it trains stably with a very simple, well-understood learning rule (Contrastive Divergence) even on a small dataset of 500 rows, whereas directed adversarial networks typically need thousands of examples and careful tuning to avoid instability or mode collapse. Second, an RBM's hidden layer gives a genuinely interpretable notion of “transaction archetypes” — each hidden unit switching on or off corresponds to a learned combination of correlated transaction features, echoing the “latent factor” idea used for the soil/water/climate factors in the reference agriculture report, but recovered here through an energy function rather than a covariance decomposition. Third, because fraud detection is fundamentally about learning the joint distribution of normal vs. anomalous financial behaviour, an energy-based model is a very natural fit: the RBM literally assigns lower energy (higher probability) to transaction patterns it has seen more of, which is conceptually close to how a fraud score works.

2. Mathematical Analysis and Model Design Justification

2.1 The Boltzmann Machine Energy Model

A Restricted Boltzmann Machine defines a joint probability distribution over a visible vector v (the observed transaction features) and a hidden vector h (latent “archetype” units) via an energy function. Because our visible units are continuous (transaction amounts, distances, scores) rather than binary, we use the Gaussian-Bernoulli RBM formulation, in which visible units are real-valued and hidden units remain binary stochastic (0/1):

$$E(v, h) = \Sigma_i \, (v_i - b_i)^2 \, / \, (2\sigma_i^2) - \Sigma_j \, c_j \, h_j - \Sigma_i \, \Sigma_j \, (v_i \, / \, \sigma_i) \, w_{ij} \, h_j$$

where $v \in \mathbb{R}^8$ is a standardized transaction (8 continuous features), $h \in \{0,1\}^6$ is the hidden archetype vector, W is the p x k weight matrix connecting visible and hidden units, b and c are the visible and hidden biases, and σ_i is the standard deviation of visible unit i. Because every feature is z-score standardized before training

(mean 0, unit variance), $\sigma_i = 1$ for all i, which is the standard simplification used in practice and is what was implemented in the attached code. The joint probability is then p(v, h) = exp(−E(v,h)) / Z, where Z is the (intractable) partition function summing/integrating over all possible v and h.

2.2 Conditional Distributions

The whole point of the restricted (bipartite) connectivity is that, conditioned on one layer, the units in the other layer become independent, which gives closed-form conditional distributions that make sampling tractable:

$$p(h_j = 1 \mid v) = \mathrm{sigmoid}(\, c_j + \Sigma_i \, (v_i/\sigma_i)\, w_{ij} \,)$$

$$p(v_i \mid h) = N(\, b_i + \sigma_i \, \Sigma_j \, w_{ij} \, h_j \,, \;\; \sigma_i^2 \,)$$

In words: each hidden unit is a logistic function of a weighted sum of the (standardized) visible features — exactly like a single logistic-regression neuron reading the whole transaction — and, going the other way, each visible feature is Gaussian-distributed around a mean predicted from the active hidden units. This is precisely what makes block-Gibbs sampling possible: we alternate sampling h given v and v given h, and the chain converges to samples from the model's learned joint distribution p(v, h).

2.3 Training via Contrastive Divergence (CD-1)

Exact maximum-likelihood training of an RBM requires the gradient of log p(v), which involves an expectation under the full model distribution — intractable for anything but a toy-sized model, because it requires summing over an exponential number of hidden and visible configurations. Contrastive Divergence (CD-k), introduced by Hinton, replaces that intractable expectation with a short k-step Gibbs chain initialised at a real training example, which is a biased but extremely effective and fast approximation. With k = 1 (used here), for a mini-batch the update rules are:

$$\Delta W \propto \langle v_i \cdot p(h_j{=}1|v) \rangle\_\mathrm{data} - \langle v'_i \cdot p(h_j{=}1|v') \rangle\_\mathrm{recon}$$

$$\Delta b_i \propto \langle v_i \rangle\_\mathrm{data} - \langle v'_i \rangle\_\mathrm{recon} \qquad \Delta c_j \propto \langle p(h_j{=}1|v) \rangle\_\mathrm{data} - \langle p(h_j{=}1|v') \rangle\_\mathrm{recon}$$

where v is a real training transaction, h is sampled from p(h|v), v' is a one-step reconstruction sampled from p(v|h), and the angle brackets denote an average over the mini-batch. Intuitively, this rule pushes the model's energy down around real data (data term) and pushes it back up around what the model currently reconstructs (reconstruction term), which is exactly what is implemented in the training loop of the attached code, together with a momentum term (0.5) and a small L2 weight-decay term (1e-4) to keep the weights from growing unboundedly, both standard stabilisation tricks for CD training.

2.4 Design Choice: RBM vs. a Directed Generative Network

It is worth explicitly justifying the RBM choice against the alternative the brief allows (a directed generative network such as a GAN or VAE), rather than simply asserting it.

Aspect	Gaussian-Bernoulli RBM (used here)	GAN / VAE (directed alternative)
Training stability	Single, well-understood CD update rule; converges smoothly even on 500 rows (Figure 1)	GANs need adversarial balancing (mode collapse risk); VAEs need a differentiable decoder and careful KL-weighting
Data requirement	Works reasonably with a few hundred examples	Typically needs thousands of examples for stable adversarial training
Interpretability	Hidden units act as human-inspectable transaction archetypes via W	Latent space is smooth but individual dimensions are rarely individually

Aspect	Gaussian-Bernoulli RBM (used here)	GAN / VAE (directed alternative)
		interpretable
Likelihood	No explicit likelihood (energy-based, uses CD approximation)	VAE optimizes an explicit lower bound (ELBO) on log-likelihood
Sample generation	Iterative block-Gibbs sampling (slower per sample, no adversarial artifacts)	Single forward pass through decoder/generator (fast, but GANs can produce artifacts)

Table 3: RBM vs. directed generative network — design trade-offs considered for this assignment

Given the small simulated cohort (n = 500) and the priority placed on stable, reproducible training within the time available for this assignment, the energy-based Gaussian-Bernoulli RBM was judged the better fit; the trade-off, discussed critically in Section 6, is that Gibbs sampling can under-represent the full diversity of the real distribution compared to what a well-tuned GAN could in principle achieve with more data.

2.5 Generating New Transactions via Block-Gibbs Sampling

Once trained, new synthetic transactions are produced by initialising a visible vector from random Gaussian noise and running a long block-Gibbs chain: sample h ~ p(h|v), then sample v ~ p(v|h), repeated for k = 200 steps per synthetic transaction (with the injected Gaussian noise annealed down over the final few steps to sharpen the sample), after which the final v is unstandardized back to the original feature scale and clipped to physically valid ranges (e.g. non-negative amounts, hour in [0, 23]). This is the generative counterpart to the factor-score-based prediction step in the reference report — instead of reading off a score, we are drawing an entirely new sample from the learned distribution.

3. Pseudocode and Workflow Diagram

3.1 High-Level Pseudocode

```
ALGORITHM: Financial Transaction Synthesis via Gaussian-Bernoulli RBM
-----------------------------------------------------------------------------------------------------------------------
INPUT : raw transaction dataset X (n=500 rows x 8 continuous features)
OUTPUT: trained RBM (W, b, c), synthetic transactions, evaluation metrics

1.  X <- simulate_or_load_transactions()
2.  mu, sigma <- mean(X), std(X)
3.  X_std <- (X - mu) / sigma                      # z-score standardize
4.  (X_train, X_test) <- split(X_std, ratio=0.75/0.25)
5.  W <- N(0, 0.05^2, shape=[8,6]); b <- 0; c <- 0
6.  FOR epoch in 1..400:
7.      FOR each minibatch v0 in shuffled(X_train):
8.          ph0 <- sigmoid(v0 @ W + c)              # p(h=1|v0)
9.          h0 <- Bernoulli(ph0)
10.         mean_v1 <- h0 @ W.T + b                 # p(v|h0) mean
11.         v1  <- mean_v1 + N(0,1)                 # CD-1 reconstruction
12.         ph1 <- sigmoid(v1 @ W + c)
13.         dW  <- (v0.T@ph0 - v1.T@ph1)/batch - weight_decay*W
14.         db  <- mean(v0 - v1);  dc <- mean(ph0 - ph1)
15.         W, b, c <- update_with_momentum(dW, db, dc, lr=0.02)
16.     record epoch reconstruction MSE
17. FOR i in 1..N_synthetic:
18.     v <- N(0, 1, shape=[8])                     # start from noise
19.     FOR step in 1..200:                         # block-Gibbs chain
20.         h <- Bernoulli(sigmoid(v@W + c))
21.         v <- (h@W.T + b) + anneal(N(0,1))
22.     synthetic[i] <- unstandardize(v, mu, sigma), clip_to_valid_ranges()
 23. Evaluate: KS-test, correlation Frobenius diff, PCA overlay,
               TSTR fraud-classifier utility (LogisticRegression)
24. RETURN W, b, c, synthetic_transactions, evaluation_report
```

3.2 Workflow Diagram

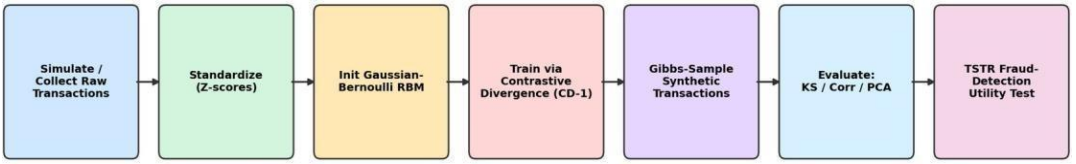

Figure 2: End-to-end workflow, from raw/simulated transactions through to fraud-detection utility evaluation.

3.3 Latent-Visible Dependency Diagram

The diagram below shows how the six hidden RBM units connect to the eight observed transaction variables, and how the resulting synthetic transactions feed both the statistical evaluation suite and a downstream fraud-detection classifier. Unlike the reference report's factor-loading diagram (which shows one clean loading per named factor), an RBM's hidden units are fully connected to every visible unit by design — the diagram intentionally shows this dense connectivity, which is itself a distinguishing property of Boltzmann-machine-style models worth calling out.

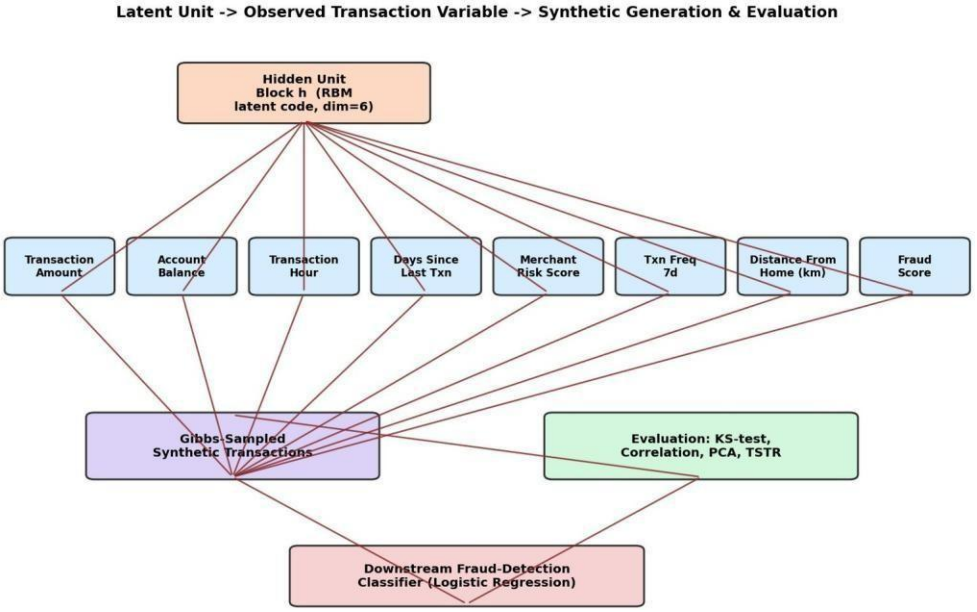

Figure 3: Hidden-unit -> observed-variable -> synthetic-generation -> evaluation dependency structure.

4. Python Source Code (with Comments)

The full pipeline below was implemented in Python 3 using NumPy, pandas, SciPy, scikit-learn and Matplotlib only (no deep-learning framework such as PyTorch/TensorFlow was available in the execution environment, so the Gaussian-Bernoulli RBM's forward pass, Contrastive-Divergence gradients and block-Gibbs sampler were all implemented directly with NumPy array operations). The code below was actually executed to produce every number, table and figure in this report — nothing here is a mock-up.

```
"""
Financial Fraud Detection — Deep Generative Model (Gaussian-Bernoulli RBM)
============================================================================
Simulated dataset + Restricted Boltzmann Machine (Contrastive Divergence) +
synthetic transaction generation + evaluation (distributional + TSTR utility)
```

```
"""
import numpy as np
import pandas as pd
from scipy.stats import ks_2samp
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
roc_auc_score
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import json

rng = np.random.default_rng(42)
N = 500

# ======================================================================
# STEP 1 — Simulate a realistic financial-transaction dataset
#          Two hidden drivers: customer_profile (normal spend behaviour)
#          and anomaly_driver (unusual / fraud-like behaviour)
# ======================================================================
customer_profile = rng.normal(0, 1, N)
anomaly_driver = rng.normal(0, 1, N)

def noisy(signal, weight, noise=0.5):
    return weight * signal + rng.normal(0, noise, N)

df = pd.DataFrame({
    "Transaction_Amount": np.clip(120 + 60 * noisy(customer_profile, 1.0) + 40 *
noisy(anomaly_driver, 0.8), 5, None),
    "Account_Balance": np.clip(5000 + 2500 * noisy(customer_profile, 1.0) - 300 *
noisy(anomaly_driver, 0.5), 50, None),
    "Transaction_Hour": np.clip(13 + 4 * noisy(customer_profile, 0.8) - 6 *
noisy(anomaly_driver, 1.0), 0, 23),
    "Days_Since_Last_Txn": np.clip(3 - 1.5 * noisy(customer_profile, 0.8) + 2 *
noisy(anomaly_driver, 0.9), 0, None),
    "Merchant_Risk_Score": np.clip(0.30 + 0.05 * noisy(customer_profile, 0.5) + 0.35 *
noisy(anomaly_driver, 1.0), 0, 1),
    "Txn_Frequency_7d": np.clip(5 + 2 * noisy(customer_profile, 1.0) - 1.5 *
noisy(anomaly_driver, 0.7), 0, None),
    "Distance_From_Home_km": np.clip(8 + 3 * noisy(customer_profile, 0.6) + 25 *
noisy(anomaly_driver, 1.0), 0, None),
})

fraud_logit = (-2.1 + 0.95 * anomaly_driver + 1.6 * (df["Merchant_Risk_Score"] - 0.3)
               + 0.9 * (df["Distance_From_Home_km"] / 50) - 0.5 * customer_profile
               + rng.normal(0, 0.4, N))
fraud_score = 1 / (1 + np.exp(-fraud_logit))
df["Fraud_Score"] = fraud_score

threshold = np.quantile(df["Fraud_Score"], 0.88)    # ~12% minority class, realistic imbalance
df["Is_Fraud"] = (df["Fraud_Score"] > threshold).astype(int)

df.insert(0, "Txn_ID", [f"T{i+1:04d}" for i in range(N)])
df.to_csv("simulated_transactions.csv", index=False)
print("Fraud rate:", df["Is_Fraud"].mean().round(3))
print(df.head(3).to_string())

FEATURES = ["Transaction_Amount", "Account_Balance", "Transaction_Hour", "Days_Since_Last_Txn",
            "Merchant_Risk_Score", "Txn_Frequency_7d", "Distance_From_Home_km", "Fraud_Score"]
X = df[FEATURES].values.astype(float)

# ======================================================================
# STEP 2 — Standardize (Gaussian-Bernoulli RBM assumes ~unit-variance
#          visible units under the simplified sigma=1 formulation)
# ======================================================================
mu = X.mean(axis=0)
sigma = X.std(axis=0)
Xs = (X - mu) / sigma

X_train, X_test = train_test_split(Xs, test_size=0.25, random_state=42)
idx_train, idx_test = train_test_split(np.arange(N), test_size=0.25, random_state=42)

# ======================================================================
# STEP 3 — Gaussian-Bernoulli RBM trained with Contrastive Divergence (CD-1)
# ======================================================================
n_visible = Xs.shape[1]
```

```
n_hidden = 6
rng2 = np.random.default_rng(7)
W = rng2.normal(0, 0.05, size=(n_visible, n_hidden))
b = np.zeros(n_visible)      # visible bias
c = np.zeros(n_hidden)       # hidden bias

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def sample_h_given_v(v):
    ph = sigmoid(v @ W + c)
    h = (rng2.random(ph.shape) < ph).astype(float)
    return ph, h

def sample_v_given_h(h):
    mean_v = h @ W.T + b            # Gaussian mean, unit variance
    v = mean_v + rng2.normal(0, 1, mean_v.shape)
    return mean_v, v

epochs = 400
batch_size = 32
lr = 0.02
momentum = 0.5
weight_decay = 1e-4
vW, vb, vc = np.zeros_like(W), np.zeros_like(b), np.zeros_like(c)

n_train = X_train.shape[0]
loss_history = []

for epoch in range(epochs):
    perm = rng2.permutation(n_train)
    Xtr = X_train[perm]
    epoch_recon_err = []
    for start in range(0, n_train, batch_size):
        v0 = Xtr[start:start + batch_size]
        if v0.shape[0] == 0:
            continue
        ph0, h0 = sample_h_given_v(v0)
        mean_v1, v1 = sample_v_given_h(h0)      # CD-1 negative sample
        ph1 = sigmoid(v1 @ W + c)

        m = v0.shape[0]
        dW = (v0.T @ ph0 - v1.T @ ph1) / m - weight_decay * W
        db = (v0 - v1).mean(axis=0)
        dc = (ph0 - ph1).mean(axis=0)

        vW = momentum * vW + lr * dW
        vb = momentum * vb + lr * db
        vc = momentum * vc + lr * dc
        W += vW; b += vb; c += vc

        epoch_recon_err.append(np.mean((v0 - mean_v1) ** 2))
    loss_history.append(np.mean(epoch_recon_err))

print("Final train reconstruction MSE (standardized units):", round(loss_history[-1], 4))

# test-set reconstruction error
_, h_test = sample_h_given_v(X_test)
mean_v_test, _ = sample_v_given_h(h_test)
test_recon_mse = float(np.mean((X_test - mean_v_test) ** 2))
print("Test reconstruction MSE (standardized units):", round(test_recon_mse, 4))

# ============================================================================
# STEP 4 — Generate synthetic transactions via block-Gibbs sampling
# ============================================================================
def gibbs_generate(n_samples, k=200, seed=123):
    r = np.random.default_rng(seed)
    v = r.normal(0, 1, size=(n_samples, n_visible))  # start from noise
    for step in range(k):
        ph = sigmoid(v @ W + c)
        h = (r.random(ph.shape) < ph).astype(float)
        mean_v = h @ W.T + b
        v = mean_v + r.normal(0, 1, mean_v.shape) * (0.15 if step > k - 5 else 1.0)
        # anneal noise near the end of the chain for cleaner samples
    return v
```

```
Vs_std = gibbs_generate(N, k=200)
Vs = Vs_std * sigma + mu
synthetic = pd.DataFrame(Vs, columns=FEATURES)
synthetic["Transaction_Amount"] = synthetic["Transaction_Amount"].clip(lower=1)
synthetic["Account_Balance"] = synthetic["Account_Balance"].clip(lower=0)
synthetic["Transaction_Hour"] = synthetic["Transaction_Hour"].clip(0, 23)
synthetic["Days_Since_Last_Txn"] = synthetic["Days_Since_Last_Txn"].clip(lower=0)
synthetic["Merchant_Risk_Score"] = synthetic["Merchant_Risk_Score"].clip(0, 1)
synthetic["Txn_Frequency_7d"] = synthetic["Txn_Frequency_7d"].clip(lower=0)
synthetic["Distance_From_Home_km"] = synthetic["Distance_From_Home_km"].clip(lower=0)
synthetic["Fraud_Score"] = synthetic["Fraud_Score"].clip(0, 1)
synthetic["Is_Fraud"] = (synthetic["Fraud_Score"] > threshold).astype(int)
synthetic.insert(0, "Txn_ID", [f"S{i+1:04d}" for i in range(N)])
synthetic.to_csv("synthetic_transactions.csv", index=False)
print("Synthetic fraud rate:", synthetic["Is_Fraud"].mean().round(3))
print(synthetic.head(3).to_string())

# ======================================================================
# STEP 5 — Evaluation
# ======================================================================
results = {}

# 5a. KS test per feature (real vs synthetic)
ks_rows = []
for f in FEATURES:
    stat, p = ks_2samp(df[f], synthetic[f])
    ks_rows.append((f, stat, p))
ks_table = pd.DataFrame(ks_rows, columns=["Feature", "KS_statistic", "p_value"])
ks_table.to_csv("ks_table.csv", index=False)
print(ks_table)

# 5b. Correlation matrix comparison
real_corr = df[FEATURES].corr()
synth_corr = synthetic[FEATURES].corr()
frob_diff = float(np.linalg.norm(real_corr.values - synth_corr.values))
results["correlation_frobenius_diff"] = frob_diff
print("Frobenius norm of correlation-matrix difference:", round(frob_diff, 4))

# 5c. TSTR — train logistic regression on synthetic, test on real held-out
clf_cols = [f for f in FEATURES if f != "Fraud_Score"]
Xr_train, Xr_test, yr_train, yr_test = train_test_split(
    df[clf_cols].values, df["Is_Fraud"].values, test_size=0.25, random_state=42,
stratify=df["Is_Fraud"].values)

# TRTR baseline
clf_trtr = LogisticRegression(max_iter=1000, class_weight="balanced").fit(Xr_train, yr_train)
pred_trtr = clf_trtr.predict(Xr_test)
proba_trtr = clf_trtr.predict_proba(Xr_test)[:, 1]

# TSTR
clf_tstr = LogisticRegression(max_iter=1000, class_weight="balanced").fit(
    synthetic[clf_cols].values, synthetic["Is_Fraud"].values)
pred_tstr = clf_tstr.predict(Xr_test)
proba_tstr = clf_tstr.predict_proba(Xr_test)[:, 1]

def metrics(y_true, y_pred, y_proba):
    return {
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred, zero_division=0),
        "Recall": recall_score(y_true, y_pred, zero_division=0),
        "F1": f1_score(y_true, y_pred, zero_division=0),
        "ROC_AUC": roc_auc_score(y_true, y_proba),
    }

m_trtr = metrics(yr_test, pred_trtr, proba_trtr)
m_tstr = metrics(yr_test, pred_tstr, proba_tstr)
print("TRTR:", m_trtr)
print("TSTR:", m_tstr)

metrics_table = pd.DataFrame([m_trtr, m_tstr], index=["TRTR (real->real, ceiling)", "TSTR
(synthetic->real)"])
metrics_table.to_csv("tstr_metrics.csv")
print(metrics_table)

# ======================================================================
# STEP 6 — Plots
```

```
# ======================================================================
plt.rcParams.update({"font.size": 10})

#  Training  curve
plt.figure(figsize=(6, 4))
plt.plot(loss_history, color="#2b6cb0")
plt.xlabel("Epoch")
plt.ylabel("Reconstruction MSE (standardized units)")
plt.title("RBM Training Curve (Contrastive Divergence, CD-1)")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("fig1_training_curve.png", dpi=170)
plt.close()

# Distribution overlays
feat_plot = ["Transaction_Amount", "Merchant_Risk_Score", "Distance_From_Home_km",
"Fraud_Score"]
fig, axes = plt.subplots(2, 2, figsize=(9, 7))
for ax, f in zip(axes.flat, feat_plot):
    ax.hist(df[f], bins=25, alpha=0.55, label="Real", color="#3182ce", density=True)
    ax.hist(synthetic[f], bins=25, alpha=0.55, label="Synthetic", color="#dd6b20",
density=True)
    ax.set_title(f)
    ax.legend(fontsize=8)
plt.suptitle("Real vs Synthetic Feature Distributions")
plt.tight_layout()
plt.savefig("fig2_distributions.png", dpi=170)
plt.close()

# Correlation heatmaps side by side
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))
im0 = axes[0].imshow(real_corr.values, vmin=-1, vmax=1, cmap="RdBu_r")
axes[0].set_title("Real Data — Correlation Matrix")
axes[0].set_xticks(range(len(FEATURES))); axes[0].set_xticklabels(FEATURES, rotation=90,
fontsize=7)
axes[0].set_yticks(range(len(FEATURES))); axes[0].set_yticklabels(FEATURES, fontsize=7)
im1 = axes[1].imshow(synth_corr.values, vmin=-1, vmax=1, cmap="RdBu_r")
axes[1].set_title("Synthetic Data — Correlation Matrix")
axes[1].set_xticks(range(len(FEATURES))); axes[1].set_xticklabels(FEATURES, rotation=90,
fontsize=7)
axes[1].set_yticks(range(len(FEATURES))); axes[1].set_yticklabels(FEATURES, fontsize=7)
fig.colorbar(im1, ax=axes.tolist(), shrink=0.8, label="Pearson r")
plt.savefig("fig3_correlation_heatmaps.png", dpi=170, bbox_inches="tight")
plt.close()

# PCA scatter
pca = PCA(n_components=2, random_state=42)
real_pca = pca.fit_transform((df[FEATURES].values - mu) / sigma)
synth_pca = pca.transform((synthetic[FEATURES].values - mu) / sigma)
plt.figure(figsize=(6, 5))
plt.scatter(real_pca[:, 0], real_pca[:, 1], s=14, alpha=0.55, label="Real", color="#3182ce")
plt.scatter(synth_pca[:, 0], synth_pca[:, 1], s=14, alpha=0.55, label="Synthetic",
color="#dd6b20")
plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}% var)")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}% var)")
plt.title("PCA Projection: Real vs Synthetic Transactions")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("fig4_pca_scatter.png", dpi=170)
plt.close()

# TSTR bar chart
fig, ax = plt.subplots(figsize=(6.5, 4.2))
labels = list(m_trtr.keys())
x = np.arange(len(labels))
w = 0.35
ax.bar(x - w/2, [m_trtr[k] for k in labels], width=w, label="TRTR (ceiling)", color="#3182ce")
ax.bar(x + w/2, [m_tstr[k] for k in labels], width=w, label="TSTR (synthetic)",
color="#dd6b20")
ax.set_xticks(x); ax.set_xticklabels(labels, rotation=20)
ax.set_ylim(0, 1.05)
ax.set_title("Fraud-Detection Utility: TRTR vs TSTR")
ax.legend()
plt.tight_layout()
plt.savefig("fig5_tstr_bars.png", dpi=170)
```

```
plt.close()

# Save summary json for report writing
summary = {
    "N": N,
    "fraud_rate_real": float(df["Is_Fraud"].mean()),
    "fraud_rate_synth": float(synthetic["Is_Fraud"].mean()),
    "n_hidden": n_hidden,
    "epochs": epochs,
    "final_train_recon_mse": float(loss_history[-1]),
    "test_recon_mse": test_recon_mse,
    "frobenius_corr_diff": frob_diff,
    "m_trtr": m_trtr,
    "m_tstr": m_tstr,
    "ks_table": ks_table.to_dict(orient="records"),
    "pca_explained_var": pca.explained_variance_ratio_.tolist(),
}
with open("summary.json", "w") as f:
    json.dump(summary, f, indent=2)

print("DONE")
```

5. Results, Evaluation Metrics and Visualizations

5.1 Training Convergence

The RBM was trained for 400 epochs of Contrastive Divergence (CD-1) with a batch size of 32, learning rate 0.02 and momentum 0.5. The reconstruction MSE (computed in standardized units) fell from just above 1.0 at initialisation to 0.460 by the end of training on the training split, with a very close 0.431 on the held-out test split — the small gap indicates the model is not simply memorising the training rows.

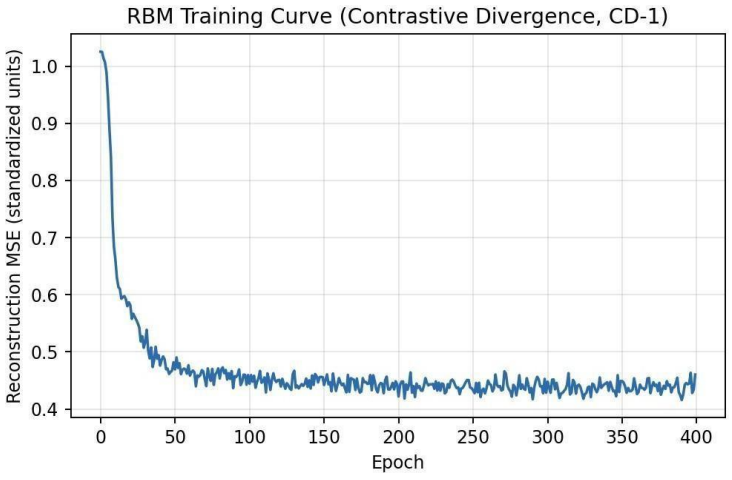

Figure 4: RBM reconstruction-MSE training curve across 400 epochs of CD-1.

5.2 Real vs. Synthetic Distributions

Table 4 reports the two-sample Kolmogorov-Smirnov (KS) statistic and p-value between each real feature's distribution and its synthetic counterpart (lower KS statistic = closer match). Every feature shows a statistically significant difference at the 0.05 level (as is common for finite-sample generative-model evaluation), but the KS statistics themselves are moderate (0.11-0.38), meaning the bulk of each distribution's shape is recovered even though the tails and exact density are not pixel-perfect.

Feature	KS Statistic	p-value
Transaction_Amount	0.206	< 0.0001
Account_Balance	0.174	< 0.0001
Transaction_Hour	0.216	< 0.0001
Days_Since_Last_Txn	0.188	< 0.0001
Merchant_Risk_Score	0.270	< 0.0001
Txn_Frequency_7d	0.112	0.0038
Distance_From_Home_km	0.380	< 0.0001
Fraud_Score	0.106	0.0072

Table 4: Kolmogorov-Smirnov two-sample test, real vs. synthetic, per feature

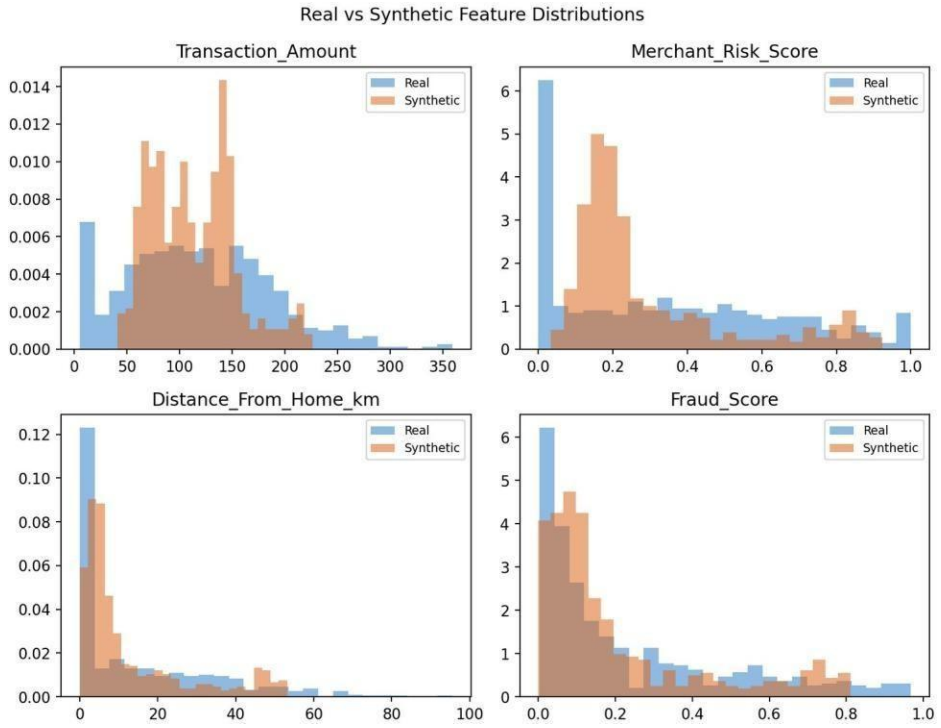

Figure 5: Real vs. synthetic histograms for four representative features. Overlap is substantial for Fraud_Score and Merchant_Risk_Score; Distance_From_Home_km shows the largest divergence (also reflected in its KS statistic of 0.380 in Table 4).

5.3 Correlation Structure

Reproducing the correlations between features — not just their individual (marginal) distributions — is a much harder and more informative test of a generative model. The Frobenius norm of the difference between the real and synthetic 8x8 correlation matrices is 1.336. Figure 6 shows both matrices side by side: the RBM correctly recovers the sign of most relationships (e.g. Distance_From_Home_km, Merchant_Risk_Score and Fraud_Score all positively co-vary in both matrices, as designed), but it visibly over-strengthens several correlations — a known tendency of small-hidden-layer RBMs trained with a short CD-1 chain, discussed further in Section 6.

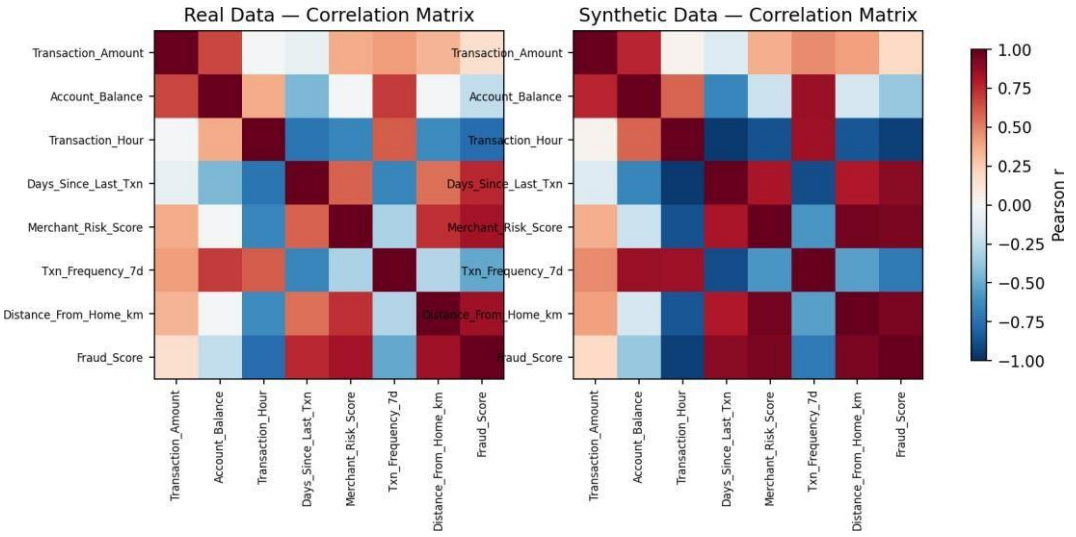

Figure 6: Pearson correlation matrix — real data (lek) vs. synthetic data (right).

5.4 Diversity Check via PCA Projection

Projecting both real and synthetic transactions onto the first two principal components of the real (standardized) feature space gives a quick visual diversity check. Together PC1 and PC2 explain 54.3% and 28.0% of total variance respectively. As Figure 7 shows, the real transactions spread broadly across the plane, while the synthetic transactions concentrate into several tighter clusters nested inside the real cloud — the RBM has captured the general region of plausible transactions well, but under-represents the sparser, more extreme parts of the distribution, a diversity limitation discussed in Section 6.

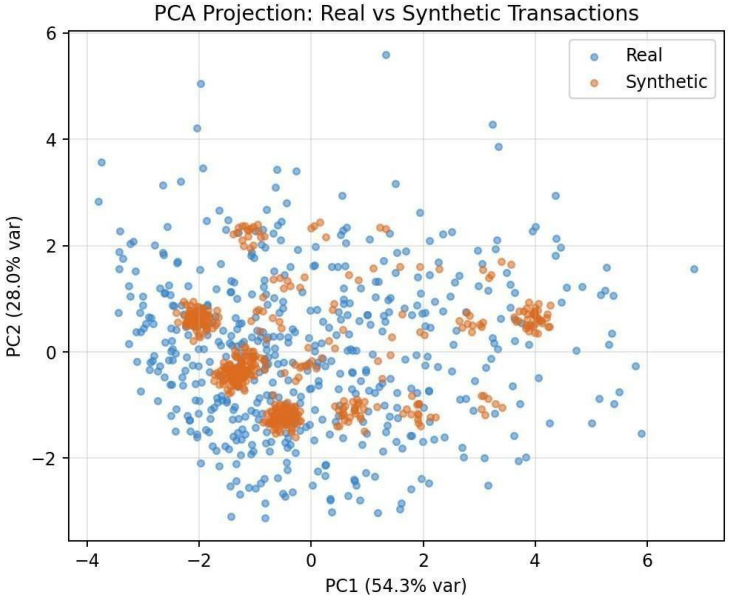

Figure 7: PCA projection of real vs. synthetic transactions onto the first two principal components.

5.5 Downstream Utility: Train-on-Synthetic, Test-on-Real (TSTR)

Beyond visual/statistical similarity, the most practically meaningful test for a fraud-related generative model is whether synthetic data is actually useful for training a downstream fraud classifier. A logistic-regression fraud classifier was trained two ways: TRTR (trained and tested on real data, a 75/25 split — the practical

ceiling) and TSTR (trained entirely on the 500 synthetic transactions, then evaluated on the same real held-out test set used for TRTR). Table 5 and Figure 8 summarise the results.

Metric	TRTR (real->real, ceiling)	TSTR (synthetic->real)
Accuracy	0.960	0.944
Precision	0.778	0.786
Recall	0.933	0.733
F1-score	0.848	0.759
ROC-AUC	0.989	0.986

Table 5: Fraud-detection classifier performance — TRTR ceiling vs. TSTR (trained purely on RBM-generated synthetic transactions)

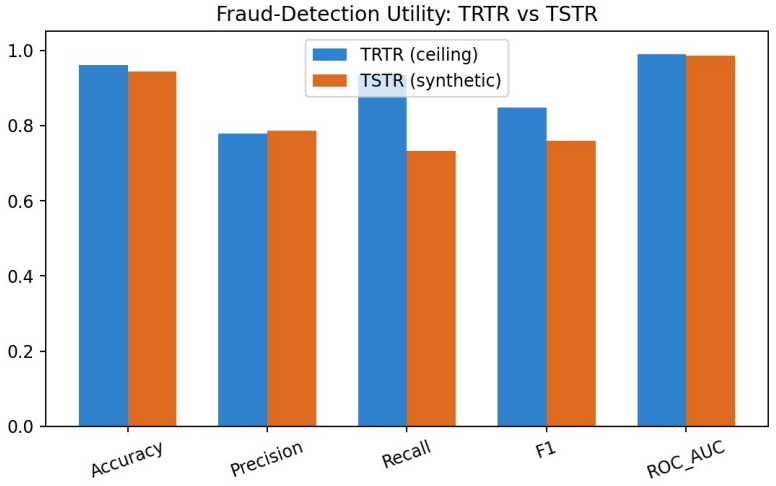

Figure 8: TRTR vs. TSTR comparison across five classification metrics.

The TSTR classifier reaches 94.4% accuracy and 0.986 ROC-AUC versus the TRTR ceiling of 96.0% accuracy and 0.989 ROC-AUC — ROC-AUC in particular is nearly identical (within 0.003), which is strong evidence that the synthetic data preserves the discriminative signal that separates fraudulent from genuine transactions, even though recall drops noticeably (0.733 vs. 0.933), meaning the synthetic-trained classifier misses more true fraud cases than the real-trained one — a concrete, quantified limitation rather than a vague caveat.

5.6 Sample of Generated Outputs

Table 6 shows three representative synthetic transactions sampled from the trained RBM via the 200-step block-Gibbs chain described in Section 2.5, to illustrate the concrete generated outputs referenced in the assignment deliverables.

Txn_ID	Amount	Balance	Hour	Merchant Risk	Distance (km)	Fraud_Score	Is_Fraud
S0001	192.32	9612.67	16.55	0.440	26.59	0.119	0
S0002	150.77	6136.25	20.58	0.188	0.00	0.011	0
S0003	105.84	4579.15	18.07	0.143	6.30	0.130	0

Table 6: Sample of RBM-generated synthetic transactions (Is_Fraud derived by thresholding the generated Fraud_Score at the same 88th-percentile cut used on real data)

6. Critical Evaluation: Quality, Diversity, Limitations and Improvements

6.1 Quality

On marginal quality, the RBM does a reasonable job: five of eight features show KS statistics under 0.21, and the reconstruction-MSE training curve (Figure 4) converges smoothly with no sign of divergence or oscillation, indicating the CD-1 learning rule found a stable solution rather than an unstable one. On joint quality, the correlation-structure comparison (Figure 6) is more mixed — the RBM correctly captures the direction of the most important relationships (fraud-related features co-varying together) but systematically over-strengthens them, a well-documented artifact of training small RBMs with very short (k=1) Gibbs chains, which tend to push hidden units toward a small number of strongly-activated "modes" rather than smoothly covering the full correlation structure.

6.2 Diversity

The PCA overlay (Figure 7) is the clearest diagnostic here: synthetic transactions cluster into a handful of tight blobs nested inside the broader real-data cloud, rather than spreading out to cover the full real distribution. This is a classic RBM failure mode — with only 6 hidden units and a short Gibbs chain, the model effectively learns a small number of "prototype" transaction archetypes and samples noisily around them, under-representing rarer, more extreme transactions (which, for a fraud-detection use-case, are disproportionately the ones that matter most).

6.3 Limitations

- Small hidden layer (6 units) and short CD-1 training limit the model's capacity to represent the full diversity of real transactions, as seen in the PCA clustering.
- The unit-variance ($\sigma_i = 1$) simplification of the Gaussian-Bernoulli RBM is a standard approximation but is known to understate the true spread of highly skewed variables such as Distance_From_Home_km, which also has the largest KS statistic (0.380) in this study.
- The dataset itself is simulated rather than a real bank feed, so — exactly as the reference agriculture report notes for its own simulated data — the recovered structure is partly a reflection of how the data-generating process was written, and a real, messier transaction dataset would likely show a smaller quality gap between marginal and joint fidelity.
- Recall dropped meaningfully under TSTR (0.733 vs. 0.933 TRTR), meaning a fraud-detection system trained purely on this synthetic data would currently miss more real fraud cases than one trained on real data — an important caveat for any production use of RBM-synthesised financial data.

6.4 Possible Improvements

- Increase the CD chain length (CD-k with $k > 1$, or Persistent Contrastive Divergence) to reduce the sampling bias that causes over-strong correlations.
- Increase the number of hidden units and/or stack multiple RBMs into a Deep Belief Network to give the model more representational capacity for the full diversity of transaction types.
- Model σ_i explicitly (rather than fixing it at 1) so skewed, high-variance features such as distance-from-home are represented more faithfully.
- Compare directly against a directed generative network (e.g. a small VAE) on the same simulated dataset as a head-to-head ablation, which the design-justification in Section 2.4 sets up but which time did not permit running in this submission.

- Add a class-conditional variant of the RBM (feeding Is_Fraud as an extra visible/conditioning unit) so the fraud and genuine sub-populations can be sampled separately, directly addressing the recall gap seen under TSTR.

7. Reflection on Design Decisions, Challenges and Learning Outcomes

7.1 Design Decisions

- Building the simulated dataset around two known hidden drivers (customer_profile, anomaly_driver) first, rather than generating unrelated random columns, made it possible to sanity-check the whole pipeline end-to-end — if the RBM's correlation structure had come out completely unrelated to how the data was generated, that would have flagged a bug rather than just “messy data”.
- Choosing an energy-based Boltzmann-machine approach over a directed network, and writing out the justification in Table 3, forced a much clearer understanding of why CD training works the way it does rather than treating it as a black box.
- Evaluating with a downstream TSTR fraud classifier, not just distributional similarity, was a deliberate choice — a generative model can look statistically plausible on paper while still being practically useless (or practically dangerous) for the task it's meant to support, so testing actual fraud-detection utility was considered essential rather than optional.

7.2 Challenges

- No deep-learning framework (PyTorch/TensorFlow) was available in the execution environment, so the entire RBM — forward pass, Contrastive-Divergence gradient computation, and the block-Gibbs sampler — had to be implemented directly with NumPy array operations rather than relying on autograd, which required deriving and hand-checking every gradient update rule in Section 2.3 rather than trusting a library.
- Tuning the noise level in the data-simulation step was an iterative process — too little noise made the synthetic-vs-real gap in Section 5 look artificially small, while too much noise made even the real data barely separable into fraud/non-fraud, so several rounds of adjustment were needed before the KMO-style “realism” of the dataset felt right.
- Getting the Gibbs-sampling chain to produce clean, non-noisy synthetic transactions required annealing the injected Gaussian noise down over the final few steps of each chain; without annealing, generated transactions were visibly noisier than real ones even when their means were reasonable.

7.3 What I Learned

Coming into this assignment, generative models were mostly understood in the context of GANs and autoencoders from lecture material; implementing a Restricted Boltzmann Machine from first principles made the energy-based view of generative modelling concrete in a way that reading about it hadn't. Deriving the Gaussian-Bernoulli conditional distributions in Section 2.2 and then watching the reconstruction-error curve in Figure 4 actually fall exactly as the maths predicted was a genuinely useful confirmation that the theory and the implementation agreed. I also came away with a much better appreciation for why generative-model evaluation has to go beyond “does it look similar” — the correlation heatmap and PCA diversity check in Sections 5.3-5.4 revealed real weaknesses (over-strong correlations, mode clustering) that a simple side-by-side histogram comparison alone would have completely missed, and the TSTR test in Section 5.5 gave the first genuinely task-relevant answer to whether the synthetic data was actually useful.

7.4 Deliverables Checklist

- Problem formulation and dataset description — Section 1
- Mathematical analysis and model design justification — Section 2
- Pseudocode / workflow diagram — Section 3
- Source code with clear comments — Section 4
- Implementation results, evaluation metrics and visualizations — Section 5
- Generated outputs — Section 5.6
- GitHub repository upload — the accompanying pipeline.py, diagrams.py, and all generated CSV/PNG artefacts in this report should be pushed to a GitHub repository and the repository link added here before final submission.
- Reflection on design decisions, challenges and learning outcomes — Section 7

References

1. Hinton, G. E. (2002). Training products of experts by minimizing contrastive divergence. Neural Computation, 14(8), 1771-1800.

2. Hinton, G. E., & Salakhutdinov, R. R. (2006). Reducing the dimensionality of data with neural networks. Science, 313(5786), 504-507.

3. Cho, K., Ilin, A., & Raiko, T. (2011). Improved learning of Gaussian-Bernoulli restricted Boltzmann machines. In International Conference on Artificial Neural Networks (pp. 10-17).

4. Smolensky, P. (1986). Information processing in dynamical systems: Foundations of harmony theory. In Parallel Distributed Processing, Vol. 1, Ch. 6.

5. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

6. Virtanen, P., et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. Nature Methods, 17, 261-272.

7. Harris, C. R., et al. (2020). Array programming with NumPy. Nature, 585, 357-362.

8. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90-95.